Implementation guide

Student side

The Way to Implement Quizzes in the App

How lesson quizzes are wired end to end in the Bilhikma mobile application: server contract, clean-architecture layers, cubit state machine, question renderers, timers, submission, result review, error handling, and testing.

Project	Bilhikma mobile (Flutter)
Feature	`lib/features/lesson_test`
Entry point	Lesson detail → Go to test → Quiz screens
Grading	Server-owned; the client never grades
Question types	7 types across choice, blanks, matching, ordering
Documented	November 2026

Contents

1. Overview and core principles
2. The server contract
3. Architecture and file map
4. Data layer: endpoints, models, repositories
5. Cubit state machine
6. Screen flow and UI structure
7. Question types and answer renderers
8. Answer editing rules
9. Timers and the timeout path
10. Submission pipeline and recovery
11. Result screen, answer review, and lesson locking
12. Error handling and localization
13. Testing
14. Adding or extending a question type
15. Product decisions and known limitations

> **One-sentence summary.** The quiz is a server-driven state machine: the API returns the quiz, an attempt with its questions and expiry, the client collects answers and submits them as IDs, and the server returns the graded result that the result screen renders verbatim.

1. Overview and core principles

Quizzes are attached to lessons. A lesson may or may not have a published quiz, the quiz may be blocking (the next lesson stays locked until it is passed), and each attempt lives on the server with its own expiry and question set. The mobile app is only a presentation and answer-collection layer.

Principles followed by the implementation

Principle	How it is enforced
Never grade on the client	No correctness logic exists in the app. `TestAnswerModel` validates only the *shape* of an answer (known IDs, counts, lengths) before sending. Score, pass/fail, per-question correctness, and awarded points all come from the submit response.
Submit IDs, not text	The payload contains option IDs, blank IDs mapped to choice IDs or typed text, right-to-left match IDs, and sentence IDs in order. No question text, no labels.
Server owns state transitions	Attempt creation, resume, expiry, attempts count, cooldown, review status, and next-lesson locking are read from responses; the app never invents them.
Dumb UI, Cubit-owned behaviour	Screens create and render; widgets display and forward taps; the cubit owns loading, selection, timers, validation keys, submission, and results.
One source of truth for failure text	`QuizErrorMapper` converts HTTP status and server reasons into stable localization keys; the cubit emits the key; the screen translates and toasts.
Resilient timer	Remaining time is recalculated from the server's `expires_at` on every tick, so backgrounding the app cannot extend an attempt.

What the feature covers

- Quiz overview card in the lesson detail screen and gated navigation into the quiz.
- Intro screen: instructions, counts, attempts, pass score, time limit, cooldown, start.
- Attempt runner: one question per page, seven question types, progress and timers.
- Submission: structural validation, single-flight guard, error mapping, recovery.
- Result: score, pass/fail, review status, retry information, optional answer review.
- Timeout: one submission at the deadline, toast, and return to the lesson screen.

2. The server contract

2.1 Endpoints

Endpoint	Method	Purpose and app usage
`/user/content/{contentId}/quiz`	GET	Quiz overview for the lesson. Loaded when the quiz screen opens.
`/user/quizzes/{quizId}/attempts`	POST	Start an attempt (empty body). Re-requesting while an attempt is open returns the same attempt — treated as a normal resume, not a new attempt.
`/user/quiz-attempts/{attemptId}/submit`	POST	Submit all answers at once as `{"answers": {questionId: answer}}`. Returns the graded result.

`/user/quizzes/{quizId}/my-attempts`	GET	Used only for recovery: when the server says the attempt was already submitted, the app finds the submitted attempt and renders its result instead of failing.

2.2 Quiz overview fields

Mapped by `LessonTestModel`.

Field	Use in the app
`id`, `content_id`, `title`, `instructions`	Identity and intro content.
`pass_score`	Pass mark shown in the intro (default 50 when absent).
`max_attempts`, `retry_cooldown_minutes`	Attempt policy display.
`time_limit_minutes`	Total attempt duration. Null/0 means untimed.
`shuffle_questions`	Kept for completeness; order comes from the attempt response.
`is_blocking`	Whether failing keeps the next lesson locked.
`show_correct_answers`	Enables the per-question review section on the result.
`is_published`	Published quizzes are the only ones the API returns to students.
`selection_mode`, `question_count`, `total_score`	Intro summary rows.
`attempts_used`, `attempts_left`	Attempts display; `null` left means unlimited.
`has_passed`	Shows the passed state; the lesson detail also reads it.
`next_attempt_at`	Cooldown; start is disabled while it is in the future.

> **Start rule.** `canStart` requires a loaded quiz, no attempt already running/submitting/result, at least one attempt left (or unlimited), no active cooldown, and `question_count > 0`.

2.3 Attempt and submit payload

The start response (and the submit response) is parsed by `QuizAttemptModel`: attempt identity, policy snapshot, questions under `answers[]`, timing, and — after submission — score and per-question results.

```
POST /user/quiz-attempts/{attemptId}/submit
{
  "answers": {
    "21": { "option_id": "o_1" },
    "22": { "option_ids": ["o_1", "o_2"] },
    "23": { "boolean": false },
    "14": { "blanks": { "b_1": "c_1" } },
    "15": { "blanks": { "b_1": "typed answer" } },
    "16": { "matches": { "right_id": "left_id" } },
    "17": { "order": ["i_1", "i_2", "i_3"] }
  }
}
```

- Unanswered questions may be omitted; they count as unanswered (zero).
- Matching is always sent as *right item → left item*.
- Blanks always send the blank ID as the key; the value is a choice ID in dropdown mode and free text in typed mode.
- Ordering requires the full list of sentence IDs.

2.4 Result fields

Field	Meaning in the UI
`score`, `max_score`, `score_percent`	Rendered as returned; no local recalculation.
`is_passed`	The only pass/fail source on the result screen.
`review_status`	`pending` shows a preliminary-result notice.
`attempts_used`, `attempts_left`, `next_attempt_at`	Retry row; no extra request needed.
`duration_seconds`	Time-spent line on the score card.
`answers[].is_correct`, `awarded_score`, `given_answer`	Per-question result and review card data.
`answers[].solution`, `explanation`	Correct answers and explanation, only when the quiz exposes them.

2.5 Status codes handled

Code	Meaning	Resulting key
401	Expired token; session refresh path applies.	session key from the shared mapper
403	Video not 100% complete, or lesson locked/out of path.	`quiz_video_incomplete`, `quiz_locked`
404	No published quiz for the lesson.	`quiz_not_found`
422	Attempts exhausted, already submitted, or invalid answers.	`quiz_attempts_exhausted`, `quiz_already_submitted`, `quiz_answers_invalid`
429	Retry cooldown active.	`quiz_cooldown`

3. Architecture and file map

The feature follows the project's two-layer clean architecture: `data` (data source, models, thin repository) and `presentation` (cubit, refactor bodies, screens, widgets). There is no domain layer.

```
lib/features/lesson_test/
  data/
    data_source/lesson_test_data_source.dart   # HTTP calls, parsing, error mapping
    models/                                    # request/response models
      lesson_test_model.dart                   # quiz overview
      quiz_attempt_model.dart                  # attempt + result + duration
      quiz_policy_model.dart                   # policy snapshot
      quiz_question_result_model.dart          # graded per-question result
      test_question_model.dart                 # one question of any type
      test_option_model.dart                   # option / word-bank item (reusable)
      test_blank_model.dart                    # blank, choices, max length
      test_match_item_model.dart               # left/right match item
      test_sentence_model.dart                 # ordering item
      test_answer_model.dart                   # local answer + submission JSON
    quiz_error_mapper.dart                     # HTTP/reason -> stable key
    repos/lesson_test_repo.dart                # thin delegation only
  presentation/
    cubit/lesson_test_cubit.dart               # state machine, timers, submission
    cubit/lesson_test_state.dart               # status, data, getters, copyWith
    screens/lesson_test_screen.dart            # provides cubit, title, toasts, pop
    refactor/
      lesson_test_args.dart                    # route argument (lessonId)
      lesson_test_body.dart                    # status -> body switch
      lesson_test_intro_body.dart              # overview/start card
      lesson_test_question_body.dart           # header + question + nav
      lesson_test_result_body.dart             # score, retry, review
      test_question_view.dart                  # per-type widget switch
      test_solution_answers.dart               # correct answers from solution
      test_blank_segments.dart                 # inline blank rendering
      test_shuffle.dart                        # display-only shuffles
    widgets/                                   # small dumb widgets per type
```

Layer responsibilities

Layer	Owns	Must not
Data source	URLs, HTTP method, body shape, JSON parsing, `QuizErrorMapper`.	Know about widgets or cubit states.
Models	`fromJson` / `toJson`, timestamps, enums, answer validation and submission JSON.	Hold UI text or navigation.
Repository	Delegate calls to the data source.	Contain business logic.
Cubit	Loading, start, selections, timers, timeout, submission, error keys, result.	Use `BuildContext`, navigate, translate text.
Screen/body/widgets	Render state and forward actions.	Call APIs, grade, map errors, compute scores.

Why a separate `lesson_test` feature

The quiz is not part of lesson detail playback: it has its own lifecycle, timers, attempt state, and result. Keeping it in its own feature prevents lesson detail from becoming a 1,000-line screen and keeps quiz state disposable when the route closes.

4. Data layer: endpoints, models, repositories

4.1 Data source

`LessonTestDataSource` uses `ApiClient(endSessionOnForbidden: false)`. A `403` from a quiz means "video incomplete" or "lesson locked", not an expired session, so it must not terminate the user's session. Every `AppException` is passed through `QuizErrorMapper`; parse failures become `quiz_unavailable` (or `quiz_unsupported_question` when the attempt contains an unknown question type).

- `fetchQuiz(contentId)` → quiz overview.
- `startAttempt(quizId)` → attempt, resuming when one is open.
- `submitAttempt(attemptId, answers)` → graded attempt.
- `fetchMyAttempts(quizId)` → list, accepting either a bare list or an `items` map.

4.2 Answer model and structural validation

`TestAnswerModel` is immutable and holds all answer shapes:

```
final List<String> optionIds;        // single/multi/most-correct
final bool? boolean;                  // true/false
final Map<String, String> blanks;    // blankId -> choiceId or text
final Map<String, String> matches;   // rightId -> leftId
final List<String> order;            // sentence ids in order
```

`toSubmissionJson(question)` validates and switches on the question type:

Type	Validation before send	JSON
Single choice / most correct	Exactly one known option.	`option_id`
Multiple choice	At least one, no duplicates, not above max selections.	`option_ids`
True/false	A boolean is present.	`boolean`
Fill-blank choice	Known blanks, known choices when the blank has choices.	`blanks`
Fill-blank typed	Known blanks, non-empty, within max length.	`blanks`
Matching	Known left/right IDs; no duplicate left unless one-to-many is allowed.	`matches`
Ordering	Full length, known and unique IDs.	`order`

Validation never judges correctness. It exists only to prevent malformed requests and to show a localized "answers invalid" message without a round trip.

4.3 Parsing details that matter

- `question_id` may be numeric; `Json.asString` normalizes it.
- Question display data lives under `options.details`; the correct answers are never present during an attempt.
- `options.default_time_seconds` becomes a per-question `Duration`; display only, the attempt expiry is authoritative.
- `fill_blank_choice` uses `details.word_bank` when choices are not embedded in each blank; word-bank items may be reusable and may include distractors.
- `details.allow_one_to_many_matching` controls whether several right items may map to the same left item.

- Ordering items arrive as `details.items`; the server order is the initial order shown to the student.
- Result answers are only treated as graded when at least one of `is_correct`, `awarded_score`, or `given_answer` is present, so an open attempt is never mistaken for a result.
- `submitted_at`, a score, a pass flag, or a non-`none` review status all mark an attempt as submitted (duplicate-submission recovery).

4.4 Repository

```
class LessonTestRepo {
  Future<LessonTestModel> fetchQuiz(int contentId);
  Future<QuizAttemptModel> startAttempt(int quizId);
  Future<QuizAttemptModel> submitAttempt(
    int attemptId,
    Map<String, Map<String, dynamic>> answers,
  );
  Future<List<QuizAttemptModel>> fetchMyAttempts(int quizId);
}
```

No logic lives here; it exists so the cubit depends on an injectable boundary and tests can supply a fake repository.

5. Cubit state machine

5.1 Status

Status	Meaning and body
`initial`, `loading`	Quiz overview is being fetched; intro skeleton.
`ready`	Overview loaded; intro card with start button.
`starting`	Start request in flight; start button shows progress.
`running`	Attempt live; question view, timers, navigation.
`submitting`	Submit in flight; nav buttons disabled.
`result`	Server result available; result body.
`failure`	Overview could not load; error view with retry.

5.2 State fields

- `quiz`, `attempt`, `result`
- `answers` — question ID to `TestAnswerModel`
- `rightOrder` — right item order for matching
- `currentIndex`, `remaining`
- `questionStartedAt`, `questionTick`
- `selectedBlankId`, `selectedPromptId`
- `errorKey`, `messageKey`, `validationErrors`
- `isStarting`, `isSubmitting`
- `didTimeOut` — set once at the deadline

5.3 Derived getters

- `canStart` — attempts left, no cooldown, questions exist, not running.
- `isRunning`, `isResult`, `hasFailed`, `isLoading`.
- `currentQuestion`, `currentAnswer`, `answerOf(id)`, `rightOrderOf(id)`.

- `questionCount`, `progress`, `isFirstQuestion`, `isLastQuestion`.
- `questionElapsed` (clock-based), `questionRemaining` (limit minus elapsed, never negative).
- `hasTimeLimit` — true when the attempt has an expiry or the quiz has a total time limit.

5.4 Public cubit API

```
loadQuiz()                       // fetch overview
startAttempt()                   // POST attempt, resume or fail
selectOption(id)                 // single/multi/most-correct
toggleBoolean(value)             // true/false
assignWord(wordId)               // word-bank -> blank
selectBlankChoice(blankId, choiceId)
setBlankText(blankId, text)
selectPrompt(leftId)             // matching selection
linkMatch(rightId)               // attach right to selected left
reorderSentences(oldIndex, newIndex)
goNext() / goPrevious()          // last question => submit()
submit({bool timedOut = false})  // single-flight submission
reportMessage(key)               // generic message toast
close()                          // stops the ticker
```

> **Single-flight guards.** Starting is guarded by `_startRequested`. Submission is guarded by `_submitRequested` plus the `submitting` / `result` statuses, so the manual "finish" action and the timeout path can never produce two concurrent submissions.

6. Screen flow and UI structure

6.1 Navigation

```
Lessons list / subject content / home
        |
        v
Lesson detail  ──(quiz summary: has_passed, is_blocking, is_locked)
        |
        |  "Go to test" (disabled while lesson is locked)
        v
LessonTestScreen (route AppRoutes.lessonTest, args: lessonId)
        |
        +-- ready       -> intro card
        +-- running     -> question body
        +-- result      -> result body (except timeout: pop back)
        |
        v  pop
Lesson detail reloads (loadLesson) so the quiz summary is fresh
Subject content reloads after the detail pops
```

6.2 Lesson detail gating

The lesson detail model carries a compact quiz summary: `QuizSummaryModel(id, isBlocking, hasPassed)`. The "Go to test" button is rendered only for lessons with a quiz and is disabled while the lesson itself is locked. When the student has passed, a "test passed" note is shown above the button. The server still enforces everything, including video completion and the 403 cases.

6.3 Screen responsibilities

Widget	Responsibility
`LessonTestScreen`	Creates the cubit, sets the app-bar title, listens for messages, handles the timeout pop.
`LessonTestBody`	Maps status to skeleton, intro, question, result, or the time-up placeholder.
`LessonTestIntroBody`	Summary rows, attempts/cooldown, start button, disabled reasons.
`LessonTestQuestionBody`	Progress/timer header, scrollable question, prev/next or finish.
`TestQuestionView`	Chooses the widget for the current question type.
`LessonTestResultBody`	Headline, score card, retry note, optional review list, back action.

6.4 Intro screen contents

- Quiz title and instructions.
- Question count, total score, pass score.
- Time limit (hidden when the quiz is untimed).
- Attempts: "N left", "unlimited", or "no attempts left".
- Cooldown: next attempt time when one is active.
- Start button, disabled when `canStart` is false; a short explanation is shown for the blocking reason.

7. Question types and answer renderers

`TestQuestionType.fromJson` maps server strings to seven enum values. Unknown types throw, and the start call surfaces `quiz_unsupported_question` instead of crashing.

Type	Server shape (`details`)	Widget	Interaction / payload
`mcq_single`	`options[]` {id, text, image_path}	`TestChoiceQuestion` single-select	Tap one option; `option_id`.
`mcq_multiple`	`options[]`, `max_selections`	`TestChoiceQuestion` multi-select	Tap toggles; max-selection limit; `option_ids`.
`true_false`	no display details	Choice-style true/false tiles	Two options mapped to a boolean; `boolean`.
`fill_blank_choice`	`blanks[]` with choices, or `word_bank[]`	`TestFillBlanksQuestion` + word bank	Select a blank, tap a word or dropdown choice; `blanks`.
`fill_blank_typed`	`blanks[]` with `max_length`	`TestTypedBlanksQuestion`	Inline text fields; `blanks` with typed text.
`matching`	`pairs.left[]`, `pairs.right[]`, one-to-many flag	`TestMatchPairsQuestion`	Select a left, tap rights; `matches` right → left.
`ordering`	`items[]` in server order	`TestOrderSentencesQuestion`	Drag to reorder; `order` with every ID.

Choice and best-answer variants

`choice_variant = best_answer` is exposed as `isBestAnswer` and changes the instruction copy ("Choose the best answer") but still submits a single `option_id`. It is treated as a single-choice question for validation.

Word bank rendering

- Each word chip has a used/disabled state derived from the answer.
- Reusable words (`is_reusable`) stay enabled after being placed, so the same word can fill several blanks.
- Non-reusable words are disabled while placed and are released when replaced.
- Distractors are ordinary words; the client does not mark them.

Ordering

The app uses the Flutter version's `ReorderableListView` callback with the already-adjusted target index; no manual `newIndex -= 1` adjustment is applied. The initial order is the server order, and the full ID list is required by the payload.

8. Answer editing rules

8.1 Selection state

Answers are kept in `state.answers` keyed by question ID and emitted with a fresh map on every change, so the UI rebuilds from cubit state and never from local widget state. Moving to another question resets the transient blank/prompt selection.

8.2 Fill-blank placement

- `assignWord(wordId)` places the word in the selected blank (or the first empty blank).
- A word already used in another blank is removed from that blank unless it is reusable.
- `selectBlankChoice` applies the same reuse rule for dropdown blanks.
- Tapping a filled blank again clears it, returning the word to the bank.
- Typed blanks are plain text fields; the model trims nothing, the server decides equivalence.

8.3 Matching

- Tap a left prompt to select it, then tap right items to link them.
- Without one-to-many support, linking a right item to a new left removes its old link, and each left keeps at most one right.
- With `allow_one_to_many_matching`, multiple rights may share a left and the prompt stays selected for fast linking; the local duplicate-left validation is skipped.
- Payload direction is always right → left.

8.4 Multiple choice

- Selecting past `max_selections` is ignored and a localized message is shown.
- The model still validates the count before sending.

8.5 Unanswered questions

Unanswered questions may be skipped; they are omitted from the payload and counted as unanswered (zero) by the server. Ordering is the exception: it always has a complete initial order and is sent even if the student never drags anything.

9. Timers and the timeout path

9.1 Total attempt timer

The authoritative deadline is the attempt's `expires_at`. A `Timer.periodic` runs once per second while the status is `running` and recalculates the difference from the current clock:

```
final Duration remaining = expiresAt.difference(DateTime.now());
if (remaining <= Duration.zero) {
  emit(state.copyWith(remaining: Duration.zero, didTimeOut: true));
  unawaited(submit(timedOut: true));
  return;
}
emit(state.copyWith(remaining: remaining));
```

Because the value is recomputed rather than decremented, backgrounding the app, screen-off time, and dropped ticks cannot extend the attempt. The ticker is stopped on submit, on result, on failure, and on cubit close.

9.2 Per-question timer

When a question carries `default_time_seconds`, elapsed time is derived from `questionStartedAt` (clock-based) and remaining is `limit - elapsed`. It is display-only: the attempt deadline remains authoritative and no auto-advance is performed on question expiry.

9.3 Timeout behaviour (current product decision)

1. The tick sets `didTimeOut` once and triggers a single submission.
2. The screen shows the localized "The test time is over" toast.
3. The quiz screen pops immediately, returning to the lesson screen that opened it.
4. The result screen is intentionally not shown for a timed-out attempt.
5. The submission still consumes the attempt on the server, so `attempts_used` increases and `attempts_left` decreases the next time the quiz is loaded.
6. If that submission fails, the ticker is not restarted (the attempt has already expired), so there is no one-request-per-second retry loop. The open expired attempt is submitted the next time it is opened.

> **Note on late answers.** Per the server contract, a submission after `expires_at` is accepted but scored zero. The client submits exactly once at the deadline and lets the server return the authoritative result or status.

9.4 Timer lifecycle

- Started after a successful start/resume and after moving between questions.
- Restarted after a failed submission only while the attempt is still live and the quiz has not timed out.
- Stopped on submit, result, timeout, and `close()`.

10. Submission pipeline and recovery

10.1 Steps

1. **Guard.** Return if a submission is already requested, in flight, or a result exists. This makes the manual finish action and the timeout path idempotent.
2. **Enter submitting.** Set `_submitRequested = true`, stop the ticker, clear messages, emit `submitting`.

3. **Build the payload.** Iterate the attempt's questions in server order, skip unanswered non-ordering questions, and serialize each answer.
4. **Validate shape.** A structural problem (unknown ID, empty blank, invalid count) is caught before the network call and surfaces `quiz_answers_invalid`; the student stays in the quiz and can fix it.
5. **POST.** Send `{"answers": ...}` to `/user/quiz-attempts/{id}/submit`.
6. **Result.** On success emit `result` with the returned attempt and clear the transient errors.

```
final Map<String, Map<String, dynamic>> payload = {};
for (final question in attempt.questions) {
  final answer = state.answerOf(question.id);
  if (!answer.isAnswered(question.type) && !question.type.isOrdering) {
    continue;
  }
  payload[question.id] = answer.toSubmissionJson(question);
}
```

10.2 Failure handling

Failure	Behaviour
401	Stay in the quiz, emit the session key as the message; the shared client handles refresh/expiry. The submission guard is released for a later retry.
Already submitted	Call `my-attempts`, find the same attempt ID, and if it is submitted emit its result instead of leaving the student stuck.
Any other mapped error	Return to `running`, keep all answers, show the localized error, and restart the ticker only if the attempt is still live and not timed out.
Parse/shape exception	Same as above with `quiz_answers_invalid`; answers are preserved.

10.3 Recovery on reopen

Starting an attempt always calls the start endpoint. If the server returns an already submitted attempt, the cubit emits the result immediately. If the attempt is open but expired, the timer detects the zero remainder and runs the timeout path. This means a student never loses an attempt because the app closed mid-quiz.

11. Result screen, answer review, and lesson locking

11.1 Score card

- Pass/fail headline from `result.isPassed` only.
- Score and maximum score from the server, with the percentage when provided; a missing percentage shows a dash instead of a computed value.
- Time spent from `duration_seconds` when present.
- Attempts left and next attempt time from the submit response; unlimited is rendered as “unlimited”, zero disables retry.
- `review_status = pending` shows a preliminary-result notice and keeps the next lesson locked.

11.2 Retry note

When the student failed and a retry is possible, the screen shows the retry message. When a cooldown is active, the next attempt time is displayed. When no attempts are left, the retry action is hidden and the message

explains it.

11.3 Answer review

Review renders only when `show_correct_answers` is true and the result contains graded answers. Each card shows:

- Question prompt (resolved from the submitted question set) and question type.
- Correct / incorrect / awaiting-review status, plus awarded and maximum points.
- The correct answer(s) extracted from `solution.details`:
  - choice types → options flagged `is_correct`;
  - true/false → localized "True" or "False";
  - dropdown blanks → correct choice text per blank;
  - typed blanks → primary accepted answer (or all accepted forms);
  - ordering → item texts in the solution order.
- Explanation from `solution['explanation']` with a fallback to the question-level explanation.

Matching solutions do not expose a per-pair breakdown in this UI; the card still shows correctness and awarded score.

11.4 Next lesson locking

There is no `next_lesson` field in the API. After a quiz, the app reloads the surrounding content so the server-side lock is reflected:

- Lesson detail reloads when the quiz route pops.
- Subject content reloads when the lesson detail pops.
- Home, view-all lessons, and archive refresh after the lesson detail route returns.

The client never unlocks a lesson itself; the effect of passing (and of a pending review) is only visible after the reload.

12. Error handling and localization

12.1 Mapping

The data source wraps `AppException` through `QuizErrorMapper`. Already-stable quiz keys pass through; otherwise:

Status	Detection	Key
403	Reason/message contains video/watch/100	`quiz_video_incomplete`
403	Reason/message contains lock/block, otherwise	`quiz_locked` / `quiz_unavailable`
404	—	`quiz_not_found`
422	attempt + exhaust/limit	`quiz_attempts_exhausted`
422	already + submit / submit	`quiz_already_submitted`
422	other validation	`quiz_answers_invalid`
429	—	`quiz_cooldown`

The mapper deliberately preserves `endsSession = false` for 403 so a locked quiz never logs the student out.

12.2 Presentation of errors

- Cubit stores the key in `errorKey` and/or `messageKey`.
- The screen's `BlocListener` translates the key and calls `AppToast.show`.
- Loading failures use the shared error view with a retry action that calls `loadQuiz` again.
- Validation errors from the server are kept as field errors and shown with the generic answers-invalid message.

12.3 Keys

Key	English copy
`quiz_locked`	This quiz is locked.
`quiz_video_incomplete`	Watch the lesson video fully to unlock the quiz.
`quiz_unavailable`	The quiz is currently unavailable.
`quiz_not_found`	No quiz was found for this lesson.
`quiz_cooldown`	You can retry after the cooldown.
`quiz_attempts_exhausted`	No attempts left.
`quiz_already_submitted`	This attempt was already submitted.
`quiz_answers_invalid`	Some answers are invalid.
`test_time_up`	The test time is over.
`quiz_unsupported_question`	This question type is not supported yet.

All keys are declared in `LangKeys` and exist in both `assets/translations/ar.json` and `en.json`. UI code uses `context.translate(LangKeys.key)` or `LangKeys.key.tr()`, never raw translation strings.

13. Testing

The feature is covered by unit tests under `test/features/lesson_test/`. There are 30 tests split by concern:

File	Coverage
`lesson_test_cubit_test.dart`	Overview load and explicit start; server question order; single/multi choice; matching direction; reusable word bank; one-to-many matching; dropdown blanks; timeout flags and a single submission; no retry loop after a failed timed-out submission; start refusals; validation failures.
`quiz_models_test.dart`	Overview parsing, all seven question types, reusable words, one-to-many matching, duration parsing, submitted detection, submission JSON keys and types, and one-to-many validation gating.
`quiz_error_mapper_test.dart`	Status-to-key mapping, 403 reason matching, 401 preservation, HTTP request shapes for overview/start/submit, and attempt list parsing (list or items map).

Commands

```
flutter test test/features/lesson_test
flutter analyze lib/features/lesson_test test/features/lesson_test
```

The fake repository in the cubit test replaces `LessonTestRepo`, so the cubit is exercised without network access. Timers are real one-second timers; the timeout tests wait for the tick and then assert that only one submission happened.

Manual verification checklist

- Lesson without a quiz shows no quiz button.
- Locked lesson disables the button; incomplete video yields `quiz_video_incomplete`.
- Start, answer all seven types, move back and forward, finish, and confirm the score matches the server.
- Force an error (airplane mode) during submit and confirm answers are preserved.
- Leave the quiz before the deadline, reopen it, and confirm the attempt resumes.
- Let a short attempt expire and confirm the toast, the pop, and the consumed attempt on the next load.
- Pass a blocking quiz; confirm the next lesson unlocks only after the reload.
- Toggle Arabic/English and confirm every quiz string is translated.

14. Adding or extending a question type

1. Add the server string and alias handling to `TestQuestionType.fromJson`; keep an explicit error for unknown values.
2. Parse the new display data inside `TestQuestionModel.fromJson` under `options.details`. Never parse a correct answer during an attempt.
3. Add the answer fields to `TestAnswerModel`, the `isAnswered` case, the structural validation, and the submission JSON branch using the exact server key.
4. Add cubit mutations for the interaction (select/toggle/assign/link/order), keeping the answer map immutable.
5. Create a small widget under `presentation/widgets` and wire it in `TestQuestionView`.
6. Add correct-answer extraction for the type in `TestSolutionAnswers`.
7. Add localization keys to both JSON files and `LangKeys` if the widget introduces new copy.
8. Extend the model tests (parsing + submission JSON) and the cubit tests (interaction + payload).
9. Run `flutter test test/features/lesson_test` and `flutter analyze`.

Do not

- Grade or compare answers in the app.
- Send question or option text instead of IDs.
- Infer pass/fail from a locally computed percentage.
- Show a correct answer before submission, even if it arrives in the payload.
- Trust a cached attempts count instead of the server response.

15. Product decisions and known limitations

Decision / limitation	Details
No result screen on timeout	By product choice, reaching the deadline submits once, shows the time-up toast, and returns to the lesson screen. The attempt is consumed on the server, but the score is not displayed in that flow.

Late submissions score zero	The server treats any submission after `expires_at` as late. The client submits exactly at the deadline and never blocks submission locally.
Per-question timer is informational	`default_time_seconds` is displayed but does not auto-advance or auto-submit; only the attempt expiry controls the quiz.
Matching review is partial	The review card shows correctness and awarded score for matching questions, but no per-pair solution list.
Pending review keeps progress locked	A written answer awaiting teacher review shows a preliminary notice and the next lesson stays locked until the server resolves it.
Client-side unlock is impossible	Lock state is refreshed by reloading content after the quiz/lesson routes pop; the app never mutates lock state directly.
Attempts count comes from the server	The local cubit does not decrement counters; it reads `attempts_used` / `attempts_left` from the quiz or the submit response.

Future improvements

- Show the graded result after a timeout on an opt-in basis.
- Per-pair solution rendering for matching questions.
- Offline queue for a timed-out submission that failed mid-request.
- Per-question time auto-advance if the backend ever enforces it.

> **Summary.** The implementation keeps a hard boundary: the app renders the quiz, collects answers, submits IDs, and displays the server's verdict. Every timer, error, and result path is derived from server data, which makes the quiz behaviour predictable, testable, and easy to extend.